PLATAFORMA MAIS ROBOTIC – 3ª EDIÇÃO
TRILHA DE DESENVOLVIMENTO MOBILE

DOCUMENTO DE ARQUITETURA E EVOLUÇÃO LÓGICA DO PROJETO DEVBURGUER

Uma mellhor Analise de coomo fio feito o projeto

Aluno: S Henrique Jr
Mediador: IA Assistente de Co-Design

São Luís – MA
Outubro de 2026

Sumário

1 Introdução e Visão Geral do Projeto **2**

2 Fase 1: O Algoritmo Primitivo – Lógica Sequencial Bloqueante **2**
2.1 Limitações Técnicas e Arquiteturais 2
2.2 O Código Inicial Completo (Esqueleto de Terminal) 2

3 Fase 2: A Transição para Manipulação Dinâmica de DOM e Estados **3**
3.1 O Banco de Dados Fictício (Array de Objetos) 3
3.2 A Árvore do DOM e o Fluxo Reativo 3
3.3 Código JavaScript Completo Refatorado 4

4 Fase 3: Arquitetura Visual e Engenharia de Layout (CSS Grid e Flexbox) **6**
4.1 Divisão de Espaço de Duas Colunas 7
4.2 Código CSS de Produção 7

5 O Git Profissional Utilizado **9**

6 Colaboração da Inteligência Artificial **9**

1 Introdução e Visão Geral do Projeto

Este documento técnico descreve o processo da evolução lógica por trás do desenvolvimento do projeto **DevBurguer**, uma aplicação de cardápio digital interativo e simulador de compras desenvolvida como o Desafio 1 da trilha de Desenvolvimento Mobile da plataforma Mais Robotic (3ª Edição).

O objetivo principal deste projeto foi consolidar os fundamentos do desenvolvimento front-end moderno (HTML5 estrutural, CSS3 responsivo e JavaScript ES6+ funcional) através de um fluxo de trabalho profissional utilizando Git e GitHub. O projeto passou por uma metamorfose arquitetural completa: saindo de um algoritmo sequencial imperativo baseado em caixas de diálogo síncronas bloqueantes (`prompt` e `alert`), evoluindo para um modelo reativo orientado a eventos com manipulação direta da árvore do DOM (*Document Object Model*), persistência de estado em memória volátil e local, e aplicação de estilos globais adaptativos através de variáveis dinâmicas de CSS.

Ao longo das seções seguintes, serão mapeadas todas as fases do desenvolvimento, contrastando os blocos de código primitivos com os atuais e destrinchando as decisões arquiteturais tomadas pela equipe de desenvolvimento para entregar um software escalável, performático e elegante.

2 Fase 1: O Algoritmo Primitivo – Lógica Sequencial Bloqueante

A primeira concepção do simulador de lanchonete utilizava o paradigma da programação imperativa puramente sequencial. O navegador funcionava apenas como um interpretador básico de comandos de texto, utilizando caixas de diálogo do sistema operacional.

2.1 Limitações Técnicas e Arquiteturais

O grande problema dessa abordagem reside no congelamento da *Main Thread* (a thread principal de execução do navegador). Funções nativas como `prompt()` e `alert()` operam em modo síncrono e bloqueante. Quando invocadas, elas impedem o motor V8 do JavaScript de executar qualquer outra tarefa paralela, travando a interface visual e impedindo interações de acessibilidade. Além disso, os dados coletados existiam apenas no escopo de execução temporário da função; assim que o usuário fechava a caixa de alerta, o estado da aplicação evaporava, caracterizando uma ausência completa de persistência.

2.2 O Código Inicial Completo (Esqueleto de Terminal)

Abaixo, documenta-se o código JavaScript primitivo utilizado como ponto de partida conceitual para o cálculo simples da compra:

```
1 function simularCompraPrimitiva() {
2     const precoFone = 150;
3     const precoCarregador = 80;
4     const precoSuporte = 50;
5
6     let qtdFone = Number(prompt("Quantos Fones Bluetooth (R$ 150,00)
    deseja?"));
```

```
    let qtdCarregador = Number(prompt("Quantos Carregadores Rapidos (R$ 80,00) deseja?"));
    let qtdSuporte = Number(prompt("Quantos Suportes Veiculares (R$ 50,00) deseja?"));

    if (isNaN(qtdFone) || qtdFone < 0) qtdFone = 0;
    if (isNaN(qtdCarregador) || qtdCarregador < 0) qtdCarregador = 0;
    if (isNaN(qtdSuporte) || qtdSuporte < 0) qtdSuporte = 0;

    let totalGeral = (qtdFone * precoFone) + (qtdCarregador * precoCarregador) + (qtdSuporte * precoSuporte);

    let resumo = "--- RESUMO DA COMPRA ---\n";
    resumo += "Total Geral: R$ " + totalGeral.toFixed(2);
    alert(resumo);
}
```

Código JavaScript Primitivo (Lógica baseada em Alertas)

3 Fase 2: A Transição para Manipulação Dinâmica de DOM e Estados

Para transformar a aplicação em uma plataforma de e-commerce real, foi realizada uma refatoração profunda baseada em estruturas de dados complexas e eventos assíncronos. Abandonou-se o HTML fixo e repetitivo em favor de uma vitrine construída via código através da leitura de um *Array de Objetos*.

3.1 O Banco de Dados Fictício (Array de Objetos)

A centralização das informações dos produtos foi movida para uma estrutura de dados maleável no JavaScript. Cada lanche ou bebida tornou-se uma entidade encapsulada contendo identificador único (`id`), nome, preço de ponto flutuante, descrição textual, caminho de arquivo para a imagem local em `assets/` e uma propriedade de metadados para controle de rótulos visuais (`badge`).

3.2 A Árvore do DOM e o Fluxo Reativo

A manipulação da árvore do DOM permite que o JavaScript leia a estrutura de tags do navegador e insira novos nós em tempo real. Sempre que um usuário interage com a aplicação, o fluxo de dados segue a seguinte engrenagem lógica:

1. O usuário dispara um evento físico (Ex: clique no botão `+ Adicionar`).
2. A função de escuta do evento captura os parâmetros identificadores do produto.
3. O *Estado* da aplicação (o array global `carrinho`) é alterado na memória.
4. A função de renderização é invocada, limpa o HTML antigo e reconstrói a lista visual com os novos dados matemáticos.

3.3 Código JavaScript Completo Refatorado

Abaixo apresenta-se o arquivo `script.js` finalizado com todas as técnicas modernas de gerenciamento de dados:

```
const PRODUTOS_CADASTRADOS = [
    {
        id: 1,
        nome: "Stack Burger",
        preco: 32.00,
        descricao: "Blend bovino de 150g, tomate, alface e muito queijo cheddar derretido com o molho especial da casa.",
        imagem: "assets/burguer.png",
        badge: "Mais Vendido"
    },
    {
        id: 2,
        nome: "Batata|Git Smash medio",
        preco: 28.00,
        descricao: "Porcao de batatas fritas super crocantes com tempero especial.",
        imagem: "assets/batata.jpeg",
        badge: ""
    },
    {
        id: 3,
        nome: "Dev Bacon",
        preco: 35.00,
        descricao: "Fatias generosas de bacon premium e molho barbecue.",
        imagem: "assets/bacon.jpeg",
        badge: "Novo"
    },
    {
        id: 4,
        nome: "Refri Co-Pole",
        preco: 6.00,
        descricao: "Lata de refrigerante trincando de gelada para acompanhar seu combo.",
        imagem: "assets/refri-coca.jpeg",
        badge: ""
    }
];

let carrinho = [];

function renderizarCardapio() {
    const gridProdutos = document.getElementById('products-grid');
    if (!gridProdutos) return;

    gridProdutos.innerHTML = '';

    PRODUTOS_CADASTRADOS.forEach(produto => {
        const card = document.createElement('div');
        card.classList.add('product-card');

        const badgeHTML = produto.badge ? `<div class="product-badge">${produto.badge}</div>` : '';

```

```
        card.innerHTML = `
            ${badgeHTML}
            <img src="${produto.imagem}" alt="${produto.nome}" class="product-img">
            <h3>${produto.nome}</h3>
            <p class="product-desc">${produto.descricao}</p>
            <div class="product-control">
                <span class="product-price">R$ ${produto.preco.toFixed(2)}</span>
                <button class="btn-add" onclick="adicionarAoCarrinho('${produto.nome}', ${produto.preco})">+ Adicionar</button>
            </div>
        `;
        gridProdutos.appendChild(card);
    });
}

function adicionarAoCarrinho(nomeItem, precoItem) {
    const itemExistente = carrinho.find(item => item.nome === nomeItem);

    if (itemExistente) {
        itemExistente.quantidade += 1;
    } else {
        carrinho.push({ nome: nomeItem, preco: precoItem, quantidade: 1 });
    }
    atualizarInterfaceCarrinho();
}

function removerDoCarrinho(nomeItem) {
    const itemIndex = carrinho.findIndex(item => item.nome === nomeItem);

    if (itemIndex > -1) {
        carrinho[itemIndex].quantidade -= 1;
        if (carrinho[itemIndex].quantidade === 0) {
            carrinho.splice(itemIndex, 1);
        }
    }
    atualizarInterfaceCarrinho();
}

function atualizarInterfaceCarrinho() {
    const containerItens = document.getElementById('cart-items');
    const elementoTotal = document.getElementById('cart-total');

    containerItens.innerHTML = '';

    if (carrinho.length === 0) {
        containerItens.innerHTML = '<p class="empty-message">Seu carrinho esta vazio...</p>';
        elementoTotal.innerText = 'R$ 0,00';
        return;
    }

    let valorTotalAcumulado = 0;

    carrinho.forEach(item => {
```

```
        const subtotalItem = item.preco * item.quantidade;
        valorTotalAcumulado += subtotalItem;

        const divItem = document.createElement('div');
        divItem.classList.add('cart-item');
        divItem.innerHTML = `
            <div class="cart-item-info">
                <h4>${item.nome}</h4>
                <span>${item.quantidade}x - R$ ${item.preco.toFixed(2)}</span>
            </div>
            <button class="btn-remove" onclick="removerDoCarrinho('${item.nome}')">Remover</button>
        `;
        containerItens.appendChild(divItem);
    });

    elementoTotal.innerText = `R$ ${valorTotalAcumulado.toFixed(2)}`;
}

function finalizarPedido() {
    if (carrinho.length === 0) {
        alert("O seu carrinho esta vazio! Adicione itens antes de fechar o pedido.");
        return;
    }
    window.location.href = "confirmacao.html";
}

function alternarTema() {
    document.body.classList.toggle('dark-mode');
    if (document.body.classList.contains('dark-mode')) {
        localStorage.setItem('tema-preferido', 'dark');
    } else {
        localStorage.setItem('tema-preferido', 'light');
    }
}

if (localStorage.getItem('tema-preferido') === 'dark') {
    document.body.classList.add('dark-mode');
}

renderizarCardapio();
```

script.js – Código de Produção Final

4 Fase 3: Arquitetura Visual e Engenharia de Layout (CSS Grid e Flexbox)

O desenvolvimento visual abandonou tabelas antiquadas para aplicar padrões modernos de layout recomendados pelo W3C.

4.1 Divisão de Espaço de Duas Colunas

Utilizou-se a propriedade `display: flex` no container principal (`.shop-container`) para fatiar o espaço horizontal da tela de maneira proporcional. A seção de produtos recebeu um fator de crescimento `flex: 3`, enquanto a barra lateral do carrinho recebeu `flex: 1`. Para melhorar a experiência do usuário em sessões longas de rolagem de tela, a barra lateral do carrinho recebeu a propriedade `position: sticky; top: 20px`, fazendo com que o painel flutue acompanhando a navegação vertical de forma inteligente.

4.2 Código CSS de Produção

Abaixo documenta-se a folha de estilos integrada contendo os seletores do Light e Dark Mode:

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Segoe UI', system-ui, sans-serif;
}

:root {
    --primary: #ff4757;
    --secondary: #2ed573;
    --dark: #2f3542;
    --bg: #f1f2f6;
    --card: #e4dada;
    --text: #2f3542;
    --border: #f1f2f6;
}

body.dark-mode {
    --dark: #9fa7ca;
    --bg: #1e222b;
    --card: #2f3542;
    --text: #f1f2f6;
    --border: #3f4654;
}

body {
    background-color: var(--bg);
    color: var(--text);
    display: flex;
    flex-direction: column;
    min-height: 100vh;
    transition: background-color 0.3s, color 0.3s;
}

.product-card, .cart-aside {
    background: var(--card);
    color: var(--text);
}

.btn-theme {
    position: fixed;
    top: 20px;
    right: 20px;
```

```
    z-index: 9999;
    background: var(--dark);
    color: white;
    border: 2px solid var(--primary);
    padding: 0.6rem 1rem;
    border-radius: 30px;
    cursor: pointer;
    font-weight: bold;
    box-shadow: 0 4px 10px rgba(0,0,0,0.2);
    transition: transform 0.2s;
}

.shop-container {
    display: flex;
    max-width: 1300px;
    margin: 2rem auto;
    gap: 2rem;
    padding: 0 1.5rem;
    width: 100%;
}

.products-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
    gap: 1.5rem;
}

.product-img {
    width: 100%;
    height: 160px;
    object-fit: cover;
    border-radius: 8px;
    margin-bottom: 1rem;
}

.cart-aside {
    flex: 1;
    min-width: 320px;
    position: sticky;
    top: 20px;
}

.success-container {
    max-width: 600px;
    margin: 4rem auto;
    background: var(--card);
    color: var(--text);
    padding: 3rem;
    border-radius: 12px;
    text-align: center;
    border-top: 6px solid var(--secondary);
    transition: background-color 0.3s, color 0.3s;
}
```

style.css – Folha de Estilos Avançada

5 O Git Profissional Utilizado

Durante todo o ciclo de engenharia de software da DevBurguer, o gerenciamento de código foi pautado no padrão internacional de ramos isolados e *Conventional Commits*.

O histórico do Git integrado exibiu com precisão as linhas de depuração verticais no VS Code: a marcação azul sinalizou blocos refatorados, a marcação verde registrou novas inserções e as setas indicaram deleções lógicas. A sincronização segura foi efetuada através da autenticação OAuth criptografada do GitHub via navegador web (*Sign in with your browser*), eliminando o tráfego de senhas planas no terminal local.

A árvore de commits seguiu a estrutura profissional documentada abaixo:

```
* 29d6522 (HEAD -> main, origin/main, origin/HEAD) part: Atualizando
   Readme e ajustes finais
* 1a34ed4 part: Add Tema claro e escuro no confirmacao.html
* f73791c part: Add Tema claro e escuro
* 1c28fc4 part: add imagem bacon e atualizando no JS
* 5fc587e feat: add conformicao html e alteranco function Finalizar
   Pedido no Js
* cf4f3ef feat: Flexbox e CSS Grid modernos para organizar o espa o
* 987a7cb Atualizando o Java
* 9ab5b6a Atualizando Html com manipula  o DOM com javaScript
* fce9be6 Adicionandon imagens para visualiza  o e pensando como
   melhorar a intera  o
* e9e47f1 moment: pequenas altera  es no index
* 653ee56 feat: Atualizando index,style e readme
```

Grafico de Linha do Tempo de Commits (git log –oneline)

6 Colaboração da Inteligência Artificial

Conforme preconizado pelos critérios avaliativos da banca examinadora da Mais Robotic, o desenvolvimento deste ecossistema contou com o suporte ativo de uma Inteligência Artificial operando estritamente como consultora de arquitetura e parceira de *Pair Programming*.

O papel da IA foi cirúrgico na transição paradigmática entre os estados bloqueantes de caixas de diálogo para a injeção reativa de nós dinâmicos via JavaScript. Ela atuou no cálculo responsivo das colunas do CSS Grid e na blindagem de segurança para interceptar erros de ponteiros nulos (`null pointers`) nas funções que rodavam de forma assíncrona entre o arquivo `index.html` e o `confirmacao.html`.

O aluno manteve total protagonismo sobre a aplicação prática: realizou o deploy físico local através do terminal integrado do VS Code, operou todos os comandos Git de sincronização na branch principal (`main`), gerenciou e capturou os arquivos de imagem locais dentro do diretório `assets/` e higienizou as entradas para blindar o sistema contra faturamento de valores negativos ou entradas de strings corrompidas (`NaN`), atingindo total compreensão cognitiva da base de código submetida.